

MongoDb: Atelier 5

Exercice 1 :

1. Création de la base donnée et de la collection "salaire" :

1 - Créer une base donnée :

use DBSalaire

```
> use DBSalaire
< switched to db DBSalaire
DBSalaire>
```

1-2 Création d'une collection :

db.salaries.insertOne({ id: "s1", nomsal: "Alami", prenomsal: "Sara", fonction: "Technicien", service: { codeser: "1", nomser: "Informatique" } });

```
> db.salaries.insertOne({
  _id: "s1",
  nomsal: "Alami",
  prenomsal: "Sara",
  fonction: "Technicien",
  service: {
    codeser: "1",
    nomser: "Informatique"
  }
});
< {
  acknowledged: true,
  insertedId: 's1'
}
DBSalaire>
```

2. Afficher les salariés triés par ordre croissant des noms :

db.salaries.find().sort({ nomsal: 1 });

```
> db.salaries.find().sort({ nomsal: 1 });
< {
  _id: 's1',
  nomsal: 'Alami',
  prenomsal: 'Sara',
  fonction: 'Technicien',
  service: {
    codeser: '1',
    nomser: 'Informatique'
  }
}
DBSalaire>
```

3. Afficher le nombre de salariés ayant la fonction "Technicien" :

db.salaries.find({ fonction: "Technicien" }).count();

```
> db.salaries.find({ fonction: "Technicien" }).count();
< 1
DBSalaire>
```

4. Supprimer le salarié ayant l'id = "s3" :

db.salaries.deleteOne({ _id: "s3" });

```
db.salaries.deleteOne({ _id: "s3" });
{
  acknowledged: true,
  deletedCount: 0
}
DBSalaire>
```

5. Afficher le nombre de salariés par fonction :

db.salaries.aggregate([{ \$group: { _id: "\$fonction", total: { \$sum: 1 } } }]);

```
> db.salaries.aggregate([
  { $group: { _id: "$fonction", total: { $sum: 1 } } }
]);
< {
  _id: 'Technicien',
  total: 1
}
DBSalaire> |
```

Exercice 2 :

1. Afficher les notaires qui travaillent à « Rabat » et trier par nom croissant :

db.terrains.find({ "notaire.adresse": "Rabat" }, { notaire: 1, _id: 0 }).sort({ "notaire.nom": 1 });

```
> db.terrains.find(
  { "notaire.adresse": "Rabat" },
  { notaire: 1, _id: 0 }
).sort({ "notaire.nom": 1 });
< {
  notaire: {
    numN: 'N1',
    nom: 'Ali',
    prenom: 'Youssef',
    adresse: 'Rabat',
    tel: '0666000000'
  }
}
```

2. Afficher le nombre total de terrains pour chaque type :

db.terrains.aggregate([{ \$group: { _id: "\$type", total: { \$sum: 1 } } }]]);

```
> db.terrains.aggregate([
  { $group: { _id: "$type", total: { $sum: 1 } } }
]);
< {
  _id: 'agricole',
  total: 1
}
{
  _id: 'lotissement',
  total: 1
}
terrainsDB> |
```

3. Afficher le prix maximum des terrains de type « agricole » :

db.terrains.aggregate([{ \$match: { type: "agricole" } }, { \$group: { _id: null, prixMax: { \$max: "\$prix" } } }]]);

```

> db.terrains.aggregate([
  { $match: { type: "agricole" } },
  { $group: { _id: null, prixMax: { $max: "$prix" } } }
]);
< {
  _id: null,
  prixMax: 200000
}
terrainsDB>

```

4. Augmenter de 1% le prix des terrains agricoles dont cin = P1 ou cin = P4 :

db.terrains.updateMany({ type: "agricole", cin: { \$in: ["P1", "P4"] } }, { \$mul: { prix: 1.01 } });

```

> db.terrains.updateMany(
  { type: "agricole", cin: { $in: ["P1", "P4"] } },
  { $mul: { prix: 1.01 } }
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
terrainsDB>

```

5. Supprimer l'adresse et le téléphone du notaire numN = N2 :

db.terrains.updateMany({ "notaire.numN": "N2" }, { \$unset: { "notaire.adresse": "", "notaire.tel": "" } });

```

> db.terrains.updateMany(
  { "notaire.numN": "N2" },
  { $unset: { "notaire.adresse": "", "notaire.tel": "" } }
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
terrainsDB>

```

